



**WYŻSZA SZKOŁA
INFORMATYKI i ZARZĄDZANIA**
z siedzibą w Rzeszowie

KOLEGIUM INFORMATYKI STOSOWANEJ

Kierunek: INFORMATYKA

Specjalność: Technologie internetowe i mobilne

Piotr Ostrowski
Nr albumu studenta: 69826

***Projekt systemu OSK-Manager do zarządzania ośrodkiem
szkolenia kierowców***

Rzeszów 2026

Spis treści

1	Analiza problemu i przegląd istniejących rozwiązań	3
1.1	Charakterystyka problemu	3
1.2	Skutki braku centralnego systemu	3
1.3	Cel projektowanego systemu	4
1.4	Przegląd istniejących rozwiązań	4
1.4.1	CarDriveManager	4
1.4.2	Portal OSK	4
1.4.3	OSKAdmin	4
1.4.4	eOSK	5
1.5	Porównanie rozwiązań	5
1.6	Wnioski z analizy	5
2	Opis biznesowy systemu	6
2.1	Wymagania funkcjonalne	6
2.2	Wymagania нефункционалне	7
3	Modelowanie systemu	8
3.1	Diagram przypadków użycia	8
3.2	Diagram klas	9
3.3	Scenariusze przypadków użycia	11
3.3.1	Scenariusz PU-01: Zaplanowanie lekcji praktycznej	11
3.3.2	Scenariusz PU-02: Ocena zakończonej lekcji	12
3.4	Diagramy aktywności	13
3.4.1	Diagram aktywności: zaplanowanie lekcji praktycznej	13
3.4.2	Diagram aktywności: ocena zakończonej lekcji	13
3.5	Diagramy sekwencji	14
3.5.1	Diagram sekwencji: zaplanowanie lekcji praktycznej	14
3.5.2	Diagram sekwencji: ocena zakończonej lekcji	15
3.6	Diagramy stanów	16
3.6.1	Diagram stanów: kurs	17
3.6.2	Diagram stanów: lekcja	17
4	Technologie i wymagania sprzętowe	19
4.1	Technologie frontendowe	19
4.2	Technologie backendowe	19
4.3	Baza danych i usługi zewnętrzne	19
4.4	Narzędzia deweloperskie	20
4.5	Wymagania sprzętowe i środowiskowe	20

5	Struktura systemu	21
5.1	Ogólna architektura	21
5.2	Warstwa frontendowa	21
5.3	Warstwa BFF	21
5.4	Warstwa backendowa	22
5.5	Opis bazy danych	22
5.6	Diagram ERD	23
5.7	Integralność danych	24
5.8	Diagram klas	25
6	Projekt GUI	26
6.1	Założenia interfejsu	26
6.2	Widok logowania	26
6.3	Dashboard	27
6.4	Panel menedżera	27
6.5	Widoki kursanta	29
6.6	Widoki instruktora	30
6.7	Wspólne elementy interfejsu	30
7	Propozycja testów	31
7.1	Testy jednostkowe	31
7.2	Testy usług backendowych	31
7.3	Testy API i kontraktu	32
7.4	Testy warstwy BFF	32
7.5	Testy manualne GUI	32
7.6	Testy regresji	33
8	Podsumowanie	34
	Spis rysunków	35
	Spis tabel	36
	Spis listingów	37
	Bibliografia	38

1. Analiza problemu i przegląd istniejących rozwiązań

Niniejszy rozdział przedstawia problem, który rozwiązuje projektowany system OSK-Manager, oraz omawia wybrane istniejące rozwiązania przeznaczone dla ośrodków szkolenia kierowców. Analiza stanowi podstawę do określenia wymagań funkcjonalnych i нефункциональных systemu.

1.1. Charakterystyka problemu

Ośrodek szkolenia kierowców prowadzi równolegle wiele procesów organizacyjnych. Musi obsługiwać kursantów, instruktorów, pojazdy, kursy, lekcje teoretyczne i praktyczne, dostępność instruktorów, płatności, dokumentację oraz harmonogram jazd. Każdy z tych obszarów jest ze sobą powiązany. Przykładowo zaplanowanie jazdy praktycznej wymaga jednoczesnego sprawdzenia kursu kursanta, dostępności instruktora, dostępności pojazdu, rodzaju zajęć i przynależności wszystkich elementów do tej samej szkoły.

W praktyce wiele szkół jazdy korzysta z rozproszonych narzędzi: arkuszy kalkulacyjnych, papierowych kart, komunikatorów, zwykłego kalendarza oraz ręcznych notatek. Takie podejście jest wystarczające przy małej liczbie kursantów, ale wraz ze wzrostem liczby instruktorów, pojazdów i kursów zwiększa ryzyko błędów. Najpoważniejsze problemy to podwójne rezerwacje instruktora lub pojazdu, brak aktualnej informacji o dostępności, trudność w monitorowaniu postępów kursanta oraz brak jednego miejsca, w którym manager widzi stan szkoły.

Projekt OSK-Manager dotyczy systemu webowego wspierającego zarządzanie szkołą jazdy. Z analizy projektu wynika, że system obejmuje role użytkowników, ośrodki szkolenia kierowców, kursantów, instruktorów, pojazdy, kursy, lekcje, wydarzenia instruktorskie, dostępność, harmonogram, płatności i oceny. System ma zatem porządkować codzienną pracę OSK oraz wymuszać najważniejsze reguły biznesowe, takie jak brak kolizji terminów, spójność szkoły oraz przypisanie lekcji do konkretnego kursu.

1.2. Skutki braku centralnego systemu

Brak jednego systemu zarządzania powoduje problemy operacyjne i jakościowe. Manager może mieć trudność z szybkim ustaleniem, który instruktor jest dostępny, który pojazd może zostać użyty do jazdy oraz ilu kursantów znajduje się na konkretnym etapie szkolenia. Instruktor może nie mieć aktualnego widoku swoich zajęć, a kursant może nie wiedzieć, ile lekcji już odbył i jakie zajęcia są przed nim.

Szczególnie istotny jest problem planowania jazd praktycznych. Lekcja praktyczna wymaga jednoczesnej dostępności instruktora i pojazdu. Jeżeli harmonogram prowadzony jest ręcznie, łatwo doprowadzić do konfliktu terminów lub zaplanowania lekcji poza godzinami pracy instruktora. Dodatkowo pojazdy wymagają monitorowania statusu, terminów przeglądów, ubezpieczenia i dostępności.

Drugim problemem jest kontrola poprawności danych. Kursant może uczestniczyć w wielu kursach, ale każda lekcja musi należeć do konkretnego kursu. Instruktor i pojazd powinni być powiązani z tą samą szkołą, której dotyczy kurs. Takie reguły trudno egzekwować w arkuszu kalkulacyjnym, natomiast system informatyczny może sprawdzać je automatycznie.

1.3. Cel projektowanego systemu

Celem OSK-Manager jest dostarczenie aplikacji wspierającej codzienne zarządzanie ośrodkiem szkolenia kierowców. System powinien umożliwiać prowadzenie bazy kursantów, instruktorów, pojazdów i kursów, a także planowanie zajęć w sposób ograniczający ryzyko konfliktów.

System powinien wspierać trzy główne grupy użytkowników. Manager szkoły powinien zarządzać ośrodkami, personelem, kursami, pojazdami i harmonogramem. Instruktor powinien mieć dostęp do swoich zajęć oraz dostępności. Kursant powinien widzieć swoje kursy i lekcje. Dodatkowo administrator powinien mieć możliwość nadzoru nad danymi systemowymi i konfiguracją.

1.4. Przegląd istniejących rozwiązań

Na rynku dostępne są systemy przeznaczone dla ośrodków szkolenia kierowców. Do porównania wybrano rozwiązania, które pokazują typowe funkcje oczekiwane w tej domenie: zarządzanie kursantami, instruktorami, pojazdami, harmonogramem, płatnościami i dokumentacją.

1.4.1. CarDriveManager

CarDriveManager jest systemem CRM i kalendarzem dla szkół jazdy. Według materiałów producenta umożliwia zarządzanie grafikami jazd, kursantami i płatnościami, a także obsługę bloków lekcyjnych oraz rozliczeń online¹. Rozwiązanie akcentuje automatyzację pracy biura oraz dostęp do informacji o kursantach i płatnościach w jednym miejscu.

Mocną stroną tego typu systemu jest kompleksowość. Obejmuje on wiele obszarów działalności OSK, od harmonogramu po płatności. Z perspektywy projektu OSK-Manager istotne jest podobne połączenie modułów operacyjnych, lecz z naciskiem na przejrzysty model ról, relacje pomiędzy szkołami, instruktorami i kursantami oraz automatyczne sprawdzanie reguł domenowych.

1.4.2. Portal OSK

Portal OSK to program do prowadzenia ośrodka szkolenia kierowców, który według opisu pozwala szybko wyszukiwać informacje o kursantach, pracownikach i pojazdach oraz wspiera prowadzenie OSK w sposób zgodny z wymaganiami organizacyjnymi². Rozwiązanie koncentruje się na centralizacji informacji potrzebnych w biurze szkoły jazdy.

Zaletą takiego systemu jest uporządkowanie danych administracyjnych. W porównaniu z OSK-Managerem można wskazać podobny cel: odejście od rozproszonych dokumentów i przeniesienie kluczowych danych do jednego systemu. OSK-Manager dodatkowo kładzie nacisk na dynamiczne wyliczanie dostępności instruktora na podstawie grafiku, wyjątków, blokad, urlopów i istniejących lekcji.

1.4.3. OSKAdmin

OSKAdmin jest aplikacją do zarządzania szkołą jazdy. Producent wskazuje panel administracyjny dla biura i właściciela, obejmujący zarządzanie kursantami, instruktorami, pojazdami oraz kalendarzem zajęć³. System ma wspierać codzienne operacje szkoły i ograniczać ręczną pracę administracyjną.

¹<https://cardrivemanager.com/start/>

²<https://www.portalosk.pl/>

³<https://oskadmin.pl/>

OSKAdmin pokazuje, że kalendarz i zarządzanie personelem są kluczowymi elementami systemu dla OSK. W OSK-Managerze podobne wymaganie realizowane jest przez moduły harmonogramu, wydarzeń instruktorskich, dostępności oraz pojazdów. Ważne jest także rozróżnienie ról użytkowników, ponieważ inne operacje wykonuje manager, inne instruktor, a inne kursant.

1.4.4. eOSK

eOSK, oferowany przez Grupę IMAGE, opisuje moduły kursantów, instruktorów, pojazdów, finansów i zgłoszeń urzędowych⁴. Wśród funkcji wymieniane są między innymi zarządzanie szkoleniami, grafikami instruktorów, flotą pojazdów oraz przypomnieniami o terminach ważności przeglądów i ubezpieczeń.

To rozwiązanie pokazuje szeroki zakres potrzeb OSK. Dla OSK-Managera istotne są zwłaszcza moduły kursów, instruktorów i pojazdów. Projektowany system powinien zapewniać podstawę do dalszej rozbudowy o finanse, raportowanie i dokumentację, ale już na poziomie MVP musi poprawnie modelować relacje pomiędzy kursantem, kursem, lekcją, instruktorem i pojazdem.

1.5. Porównanie rozwiązań

Tab. 1.1. Porównanie OSK-Manager z wybranymi rozwiązaniami

Kryterium	CarDrive Manager	Portal OSK	OSKAdmin / eOSK	OSK-Manager
Kursanci i kursy	Obsługa kursantów i postępów	Baza kursantów i informacji OSK	Moduły kursantów i szkoleń	Kursanci, kursy, typy kursów, status uczestnictwa i PKK
Instruktorzy	Grafik instruktorów	Dane pracowników	Grafiki i dostępność instruktorów	Instruktorzy, kwalifikacje, przypisania do OSK i dostępność
Pojazdy	Uwzględnienie pojazdów w grafiku	Informacje o pojazdach	Flota, przeglądy i ubezpieczenia	Pojazdy przypisane do OSK, status, zdjęcia i wykorzystanie w jazdach
Harmonogram	Kalendarz jazd i bloków lekcyjnych	Organizacja pracy ośrodka	Kalendarz zajęć i grafiki	Harmonogram wyliczany z lekcji, wydarzeń i dostępności
Reguły biznesowe	Automatyzacja procesów OSK	Centralizacja danych	Szeroki zakres modułów OSK	Spójność OSK, brak kolizji, lekcje tylko w ramach kursów

1.6. Wnioski z analizy

Analiza istniejących rozwiązań pokazuje, że systemy dla OSK skupiają się na kilku powtarzalnych obszarach: kursantach, instruktorach, pojazdach, harmonogramie, płatnościach i dokumentacji. Oznacza to, że projekt OSK-Manager odpowiada na realną potrzebę rynkową, a jego zakres jest zgodny z typowymi wymaganiami szkół jazdy.

Najważniejszym wyróżnikiem projektowanego systemu powinno być poprawne odwzorowanie reguł domenowych. System musi pilnować, aby lekcje należały do kursów, jazdy praktyczne miały przypisany pojazd, instruktorzy i pojazdy nie byli rezerwowani w tym samym czasie oraz aby dane dotyczyły właściwej szkoły. W kolejnych rozdziałach wymagania te zostaną rozwinięte w postaci wymagań funkcjonalnych, niefunkcjonalnych oraz modeli systemu.

⁴<https://www.grupaimage.eu/p696/eosk-program-do-zarzadzania-osk.html>

2. Opis biznesowy systemu

OSK-Manager jest aplikacją webową służącą do zarządzania ośrodkiem szkolenia kierowców. System obejmuje obsługę szkół jazdy, użytkowników, kursantów, instruktorów, kursów, lekcji, pojazdów, dostępności, harmonogramu oraz wybranych danych rozliczeniowych. Poniżej przedstawiono wymagania funkcjonalne i нефункционалне wynikające z zakresu projektu.

2.1. Wymagania funkcjonalne

Wymagania funkcjonalne określają funkcje, które powinien udostępniać system OSK-Manager. Podstawowym zadaniem systemu jest obsługa kont użytkowników oraz dostępu do aplikacji. Użytkownik powinien mieć możliwość zalogowania się za pomocą adresu e-mail i hasła, sprawdzenia aktualnej sesji, odświeżenia jej oraz wylogowania. System powinien rozróżniać role kursanta, instruktora, managera i administratora, ponieważ zakres dostępnych funkcji zależy od odpowiedzialności danej osoby. Uprawnieni użytkownicy powinni mieć możliwość rejestrowania kursantów i instruktorów, a każdy użytkownik powinien móc zarządzać podstawowymi danymi swojego profilu, takimi jak telefon, opis czy avatar.

Istotną częścią systemu jest zarządzanie ośrodkami szkolenia kierowców. Manager powinien mieć możliwość tworzenia, edycji, listowania i usuwania własnych ośrodków, a także przypisywania do nich kursantów oraz instruktorów. System powinien umożliwiać przeglądanie listy kursantów, szczegółów kursanta, notatek oraz numeru PKK. W przypadku instruktorów powinien obsługiwać dodawanie, edycję, usuwanie i przeglądanie osób przypisanych do danego ośrodka. Dodatkowo system powinien przechowywać informacje o kwalifikacjach instruktorów, czyli o typach kursów, które mogą oni prowadzić.

System powinien wspierać obsługę zasobów ośrodka, przede wszystkim pojazdów i typów kursów. Manager powinien mieć możliwość dodawania, edycji, usuwania oraz zmiany statusu pojazdów, a także dodawania ich zdjęć. Aplikacja powinna również udostępniać słownik typów kursów, na przykład odpowiadających kategoriom prawa jazdy. Na tej podstawie możliwe powinno być tworzenie i edytowanie kursów powiązanych z kursantem, ośrodkiem oraz typem kursu. System powinien obsługiwać przypisanie kursanta do kursu oraz zmianę statusu uczestnictwa.

Jednym z kluczowych obszarów funkcjonalnych jest harmonogram zajęć. System powinien umożliwiać tworzenie, edycję i przeglądanie lekcji teoretycznych oraz praktycznych. W przypadku lekcji praktycznej wymagane powinno być przypisanie pojazdu. Harmonogram powinien być dostępny w różnych ujęciach, między innymi z perspektywy szkoły, instruktora i kursanta. Instruktor powinien mieć możliwość definiowania domyślnych godzin pracy, wyjątków dziennych, blokad czasu oraz urlopów. Na podstawie tych danych, a także na podstawie zaplanowanych lekcji, system powinien obliczać wolne terminy instruktora. Aplikacja powinna blokować planowanie lekcji nakładających się czasowo dla tego samego instruktora lub pojazdu. Powinna także pozwalać na tworzenie wydarzeń instruktorskich, takich jak jazda albo teoria, oraz przypisywanie do nich kursantów.

System powinien obejmować również wybrane funkcje rozliczeniowe i informacyjne. Dla kursu powinny być przechowywane informacje o planie płatności oraz dokonanych wpłatach. Kursant powinien mieć możliwość oceny zakończonej jazdy, jeżeli jest do tego uprawniony. Panel kursanta powinien zapewniać podgląd jego własnych kursów oraz zaplanowanych lekcji, natomiast panel instruktora powinien prezentować jego harmonogram i dostępność.

2.2. Wymagania niefunkcjonalne

Wymagania niefunkcjonalne określają oczekiwane cechy jakościowe systemu OSK-Manager. Jednym z najważniejszych wymagań jest bezpieczeństwo dostępu. Korzystanie z chronionych funkcji systemu powinno wymagać poprawnej sesji użytkownika, a operacje powinny być ograniczane zgodnie z rolą użytkownika oraz jego powiązaniem z konkretnym ośrodkiem szkolenia kierowców. Manager powinien zarządzać wyłącznie szkołami, których jest właścicielem, natomiast instruktor i kursant powinni widzieć jedynie dane związane z ich kontem oraz przypisanym ośrodkiem. Token odświeżający powinien być przechowywany w ciasteczku HTTP-only, a klucze serwisowe wykorzystywane przez backend nie powinny być dostępne po stronie klienta.

System powinien zapewniać spójność danych domenowych. Lekcja powinna być powiązana z właściwym kursem, kurs z odpowiednim ośrodkiem i typem kursu, a pojazdy oraz instruktorzy z właściwą szkołą. Dane przekazywane do API powinny być walidowane po stronie backendu niezależnie od walidacji formularzy w interfejsie użytkownika. Szczególne znaczenie ma zapobieganie podwójnym rezerwacjom, dlatego system powinien wykrywać konflikty terminów dla instruktorów i pojazdów oraz blokować operacje, które naruszałyby harmonogram.

Wymaganiem jakościowym jest także utrzymywalność architektury. Backend powinien zachowywać przejrzysty podział na trasy, kontrolery, serwisy, schematy walidacji oraz warstwę dostępu do danych. Frontend powinien być modularny i dzielić logikę na strony, komponenty, kompozycje, typy oraz warstwę pośredniczącą w komunikacji z backendem. Taki podział ułatwia rozwój aplikacji, testowanie oraz wprowadzanie zmian bez naruszania pozostałych części systemu.

Interfejs użytkownika powinien być responsywny i czytelny. Aplikacja powinna być używalna zarówno na typowych ekranach komputerów, jak i urządzeń mobilnych. Dane powinny być prezentowane w sposób zrozumiały dla pracownika biura, instruktora oraz kursanta. System powinien również zwracać jednoznaczne komunikaty błędów w sytuacjach takich jak brak uprawnień, błędne dane wejściowe, konflikt terminu lub brak wymaganego zasobu.

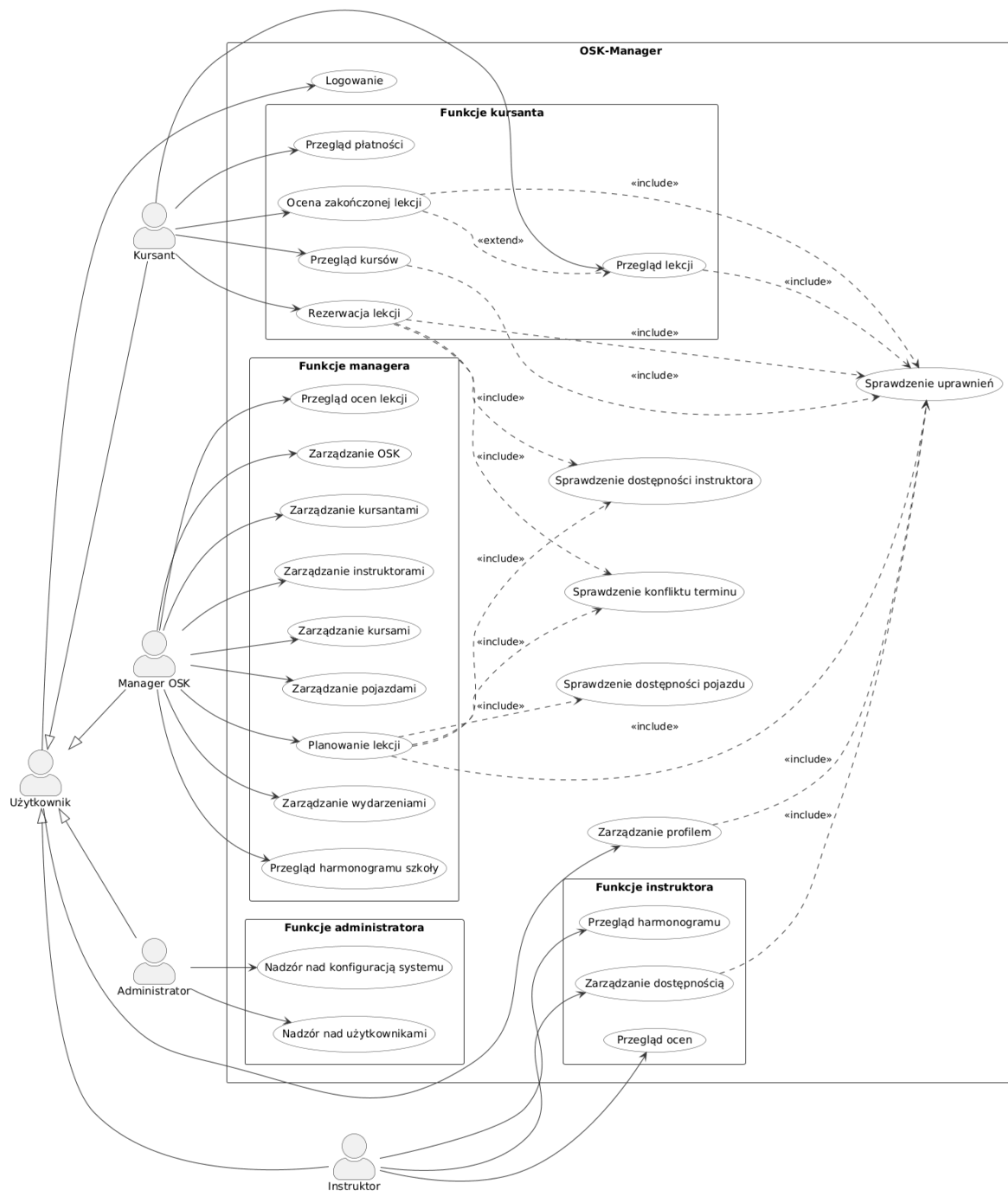
System powinien być projektowany z myślą o testowalności i skalowalności. Kluczowe reguły domenowe, zwłaszcza dotyczące dostępności instruktorów i konfliktów harmonogramu, powinny być możliwe do testowania automatycznego. Model danych powinien pozwalać na obsługę wielu szkół, instruktorów i kursantów bez konieczności zmiany podstawowej struktury systemu. Operacje harmonogramu powinny być interpretowane w lokalnym czasie Polski, zgodnie z założeniem strefy Europe/Warsaw.

3. Modelowanie systemu

3.1. Diagram przypadków użycia

Diagram przypadków użycia przedstawia system z perspektywy jego użytkowników. Nie opisuje on szczegółów technicznych, struktury bazy danych ani kolejnych ekranów aplikacji, lecz pokazuje, jakie główne cele mogą realizować poszczególni aktorzy. W przypadku systemu OSK-Manager diagram obejmuje cały system na poziomie biznesowym, ponieważ dokumentowana aplikacja jest spójnym narzędziem do zarządzania ośrodkiem szkolenia kierowców.

Na diagramie wyróżniono trzy główne role użytkowników: kursanta, instruktora oraz managera OSK. Kursant korzysta z funkcji związanych z własnym kursem i lekcjami. Instruktor zarządza dostępnością oraz realizuje zajęcia przypisane do jego grafiku. Manager OSK odpowiada za część organizacyjną, czyli kursy, osoby przypisane do ośrodka, pojazdy, harmonogram oraz płatności.



Rys. 3.1. Diagram przypadków użycia systemu OSK-Manager

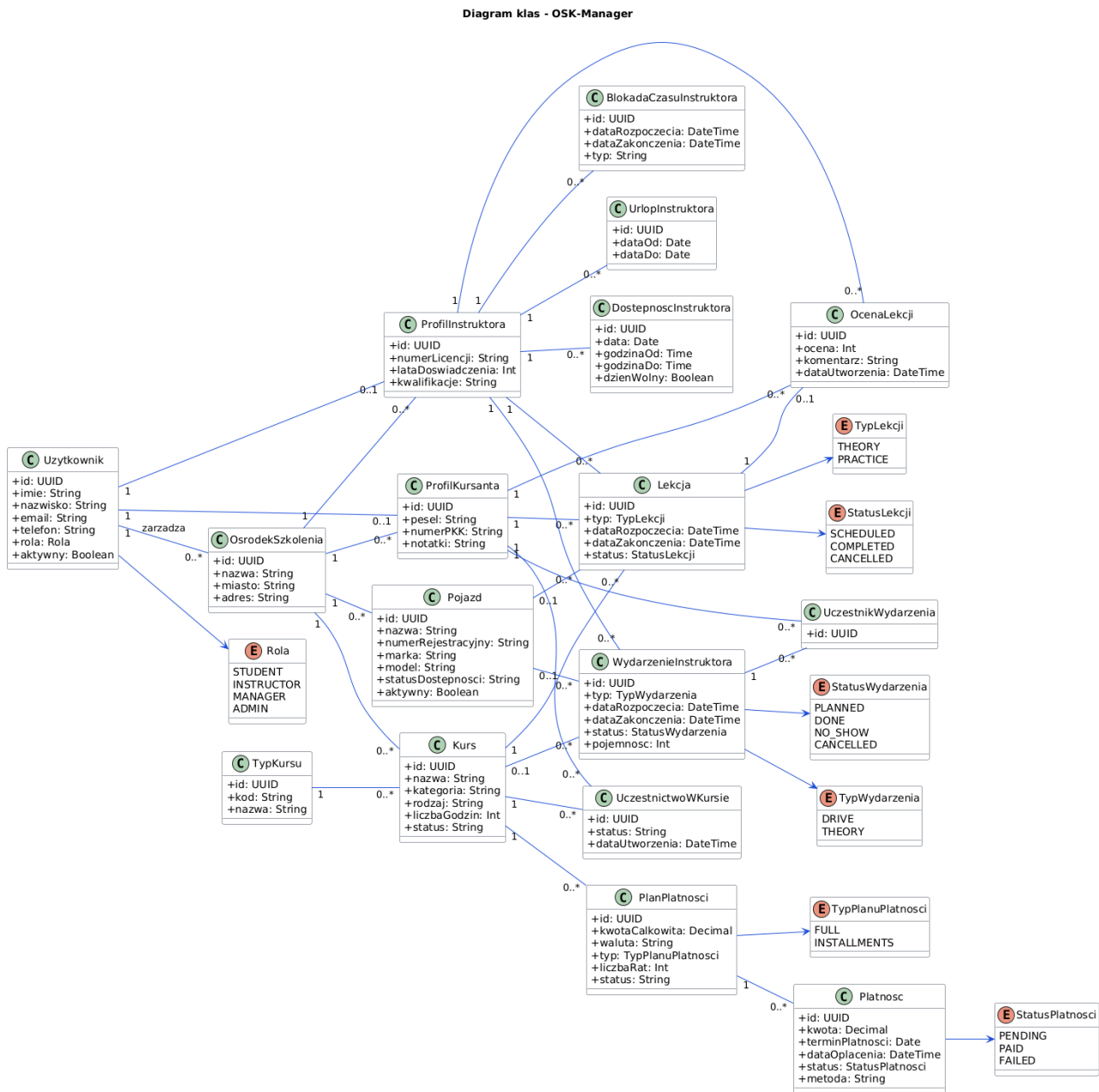
Źródło: Opracowanie własne

Przedstawiony diagram celowo pomija szczegóły implementacyjne, takie jak konkretne endpointy API, formularze lub komponenty interfejsu. Takie elementy nie są celem diagramu przypadków użycia. W dokumentacji pełnią one rolę materiału pomocniczego dla dalszych części, przede wszystkim scenariuszy przypadków użycia oraz diagramów aktywności i sekwencji.

3.2. Diagram klas

Diagram klas przedstawia uproszczony model domenowy systemu OSK-Manager oraz najważniejsze relacje między obiektami. Model koncentruje się na klasach potrzebnych do opisania głównego

procesu biznesowego: zarządzania szkołą jazdy, kursami, lekcjami, instruktorami, pojazdami i płatnościami. Na diagramie celowo pominięto część klas pomocniczych oraz szczegóły techniczne takie jak kontrolery, serwisy, endpointy API i komponenty interfejsu użytkownika.



Rys. 3.2. Diagram klas systemu OSK-Manager

Źródło: Opracowanie własne

Klasa *Użytkownik* reprezentuje wszystkie osoby korzystające z systemu, a ich uprawnienia wynikają z przypisanej roli. Ośrodek szkolenia jest centralnym elementem modelu, ponieważ grupuje kursy, pojazdy i użytkowników. Kurs składa się z lekcji, a lekcja praktyczna może być powiązana z pojazdem. Dostępność instruktora wspiera planowanie terminów, natomiast płatności opisują podstawową część rozliczeniową kursu.

3.3. Scenariusze przypadków użycia

Poniżej przedstawiono dwa scenariusze przypadków użycia opisujące kluczowe procesy systemu OSK-Manager. Pierwszy dotyczy pracy managera OSK przy planowaniu lekcji praktycznej, natomiast drugi opisuje działanie kursanta po zakończeniu lekcji.

3.3.1. Scenariusz PU-01: Zaplanowanie lekcji praktycznej

Aktor główny: Manager OSK.

Cel: Utworzenie lekcji praktycznej dla kursanta z przypisanym instruktorem, pojazdem oraz terminem.

Warunki początkowe:

- Manager OSK jest zalogowany w systemie.
- W systemie istnieje aktywny kursant przypisany do wybranego kursu.
- W systemie istnieje instruktor oraz pojazd należący do tego samego ośrodka OSK.
- Instruktor ma zdefiniowaną dostępność w harmonogramie.

Scenariusz główny:

1. Manager OSK otwiera moduł harmonogramu.
2. System wyświetla kalendarz zajęć oraz dostępne terminy.
3. Manager wybiera kursanta i kurs, dla którego ma zostać zaplanowana lekcja.
4. Manager wybiera typ lekcji jako jazdę praktyczną.
5. System prezentuje dostępnych instruktorów oraz pojazdy powiązane z danym OSK.
6. Manager wybiera instruktora, pojazd, datę oraz godzinę rozpoczęcia lekcji.
7. System sprawdza, czy instruktor i pojazd są dostępni w wybranym terminie.
8. System sprawdza, czy lekcja należy do właściwego kursu oraz tego samego ośrodka OSK.
9. Manager zatwierdza utworzenie lekcji.
10. System zapisuje lekcję i aktualizuje harmonogram.

Scenariusze alternatywne:

- Jeżeli instruktor jest niedostępny, system blokuje zapis i informuje managera o konflikcie terminu.
- Jeżeli pojazd jest już przypisany do innej lekcji w tym samym czasie, system blokuje zapis i wymaga wyboru innego pojazdu lub terminu.
- Jeżeli wybrany kursant, instruktor lub pojazd nie należy do tego samego OSK, system odrzuca operację.

Warunki końcowe:

- Lekcja praktyczna zostaje zapisana w systemie.
- Harmonogram kursanta i instruktora zostaje zaktualizowany.
- Wybrany pojazd jest oznaczony jako zajęty w czasie trwania lekcji.

3.3.2. Scenariusz PU-02: Ocena zakończonej lekcji

Aktor główny: Kursant.

Cel: Wystawienie oceny zakończonej lekcji praktycznej.

Warunki początkowe:

- Kursant jest zalogowany w systemie.
- Kursant ma przypisaną lekcję praktyczną.
- Lekcja ma status zakończony.
- Lekcja nie została wcześniej oceniona przez kursanta.

Scenariusz główny:

1. Kursant otwiera widok swoich lekcji.
2. System wyświetla listę lekcji przypisanych do kursanta.
3. Kursant wybiera zakończoną lekcję praktyczną.
4. System wyświetla szczegóły lekcji oraz formularz oceny.
5. Kursant wybiera ocenę i opcjonalnie wpisuje komentarz.
6. Kursant zatwierdza formularz.
7. System sprawdza, czy lekcja jest zakończoną lekcją praktyczną i czy nie posiada jeszcze oceny.
8. System zapisuje ocenę lekcji.
9. System wyświetla potwierdzenie zapisania oceny.

Scenariusze alternatywne:

- Jeżeli lekcja nie jest zakończona, system nie pozwala wystawić oceny.
- Jeżeli lekcja jest lekcją teoretyczną, system nie udostępnia formularza oceny.
- Jeżeli lekcja została już oceniona, system pokazuje istniejącą ocenę i blokuje ponowne wystawienie oceny.
- Jeżeli kursant próbuje ocenić lekcję innego kursanta, system odmawia dostępu.

Warunki końcowe:

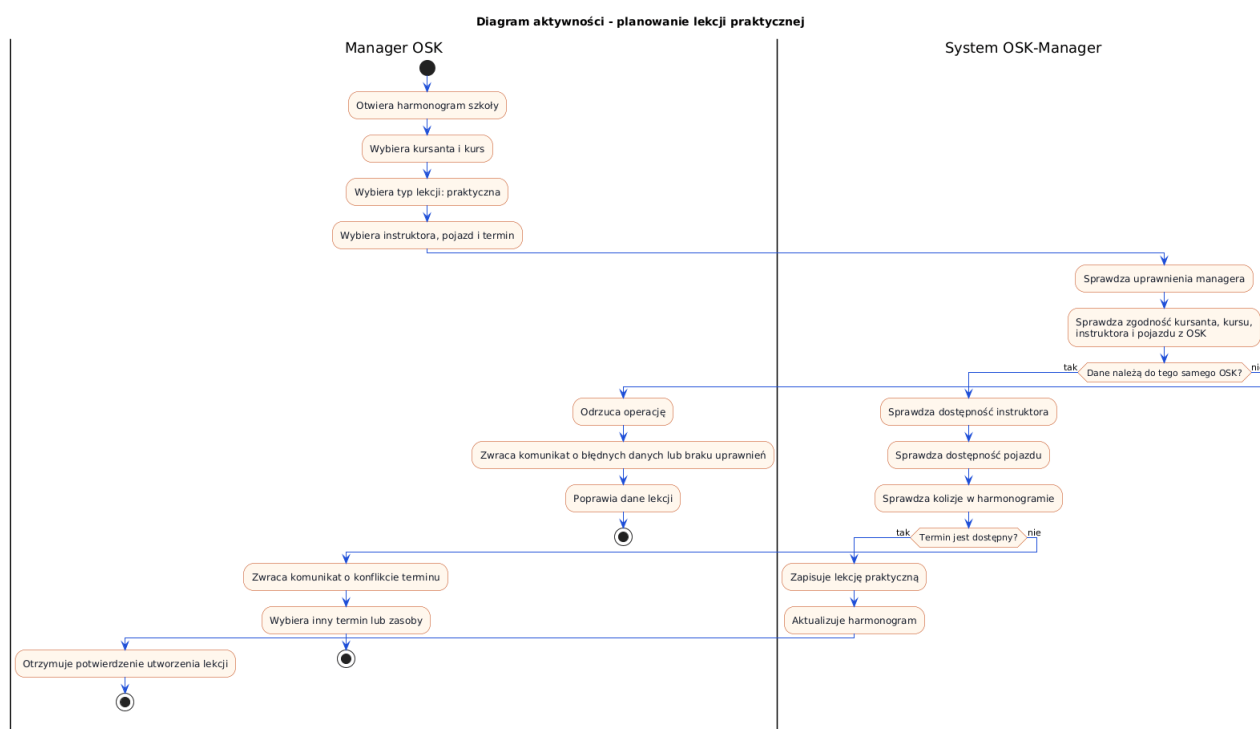
- Ocena lekcji zostaje zapisana w systemie.
- Kursant może zobaczyć zapisaną ocenę w szczegółach lekcji.
- Instruktor i manager OSK mogą wykorzystać ocenę jako informację zwrotną dotyczącą jakości szkolenia.

3.4. Diagramy aktywności

Diagramy aktywności przedstawiają kolejność działań wykonywanych przez użytkowników oraz system w ramach wybranych procesów biznesowych. W dokumentacji systemu OSK-Manager przygotowano dwa diagramy odpowiadające scenariuszom przypadków użycia: zaplanowaniu lekcji praktycznej oraz ocenie zakończonej lekcji.

3.4.1. Diagram aktywności: zaplanowanie lekcji praktycznej

Pierwszy diagram przedstawia proces tworzenia lekcji praktycznej przez managera OSK. Szczególną uwagę zwrócono na kontrolę dostępności instruktora i pojazdu, ponieważ są to warunki konieczne do poprawnego zapisania zajęć w harmonogramie.

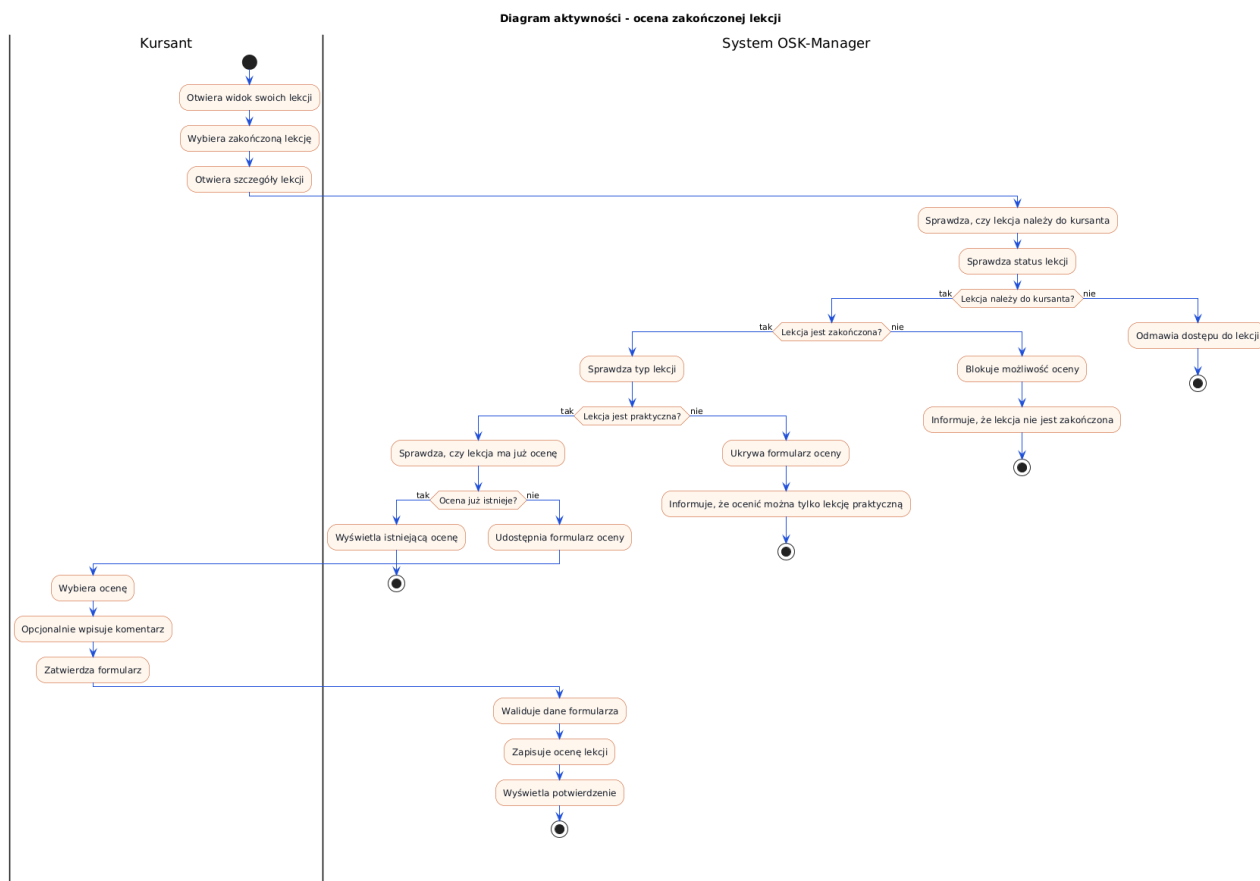


Rys. 3.3. Diagram aktywności procesu planowania lekcji praktycznej

Źródło: Opracowanie własne

3.4.2. Diagram aktywności: ocena zakończonej lekcji

Drugi diagram przedstawia proces wystawienia oceny lekcji przez kursanta. Proces obejmuje sprawdzenie, czy lekcja została zakończona, czy jest lekcją praktyczną oraz czy nie została wcześniej oceniona.



Rys. 3.4. Diagram aktywności procesu oceny zakończonej lekcji

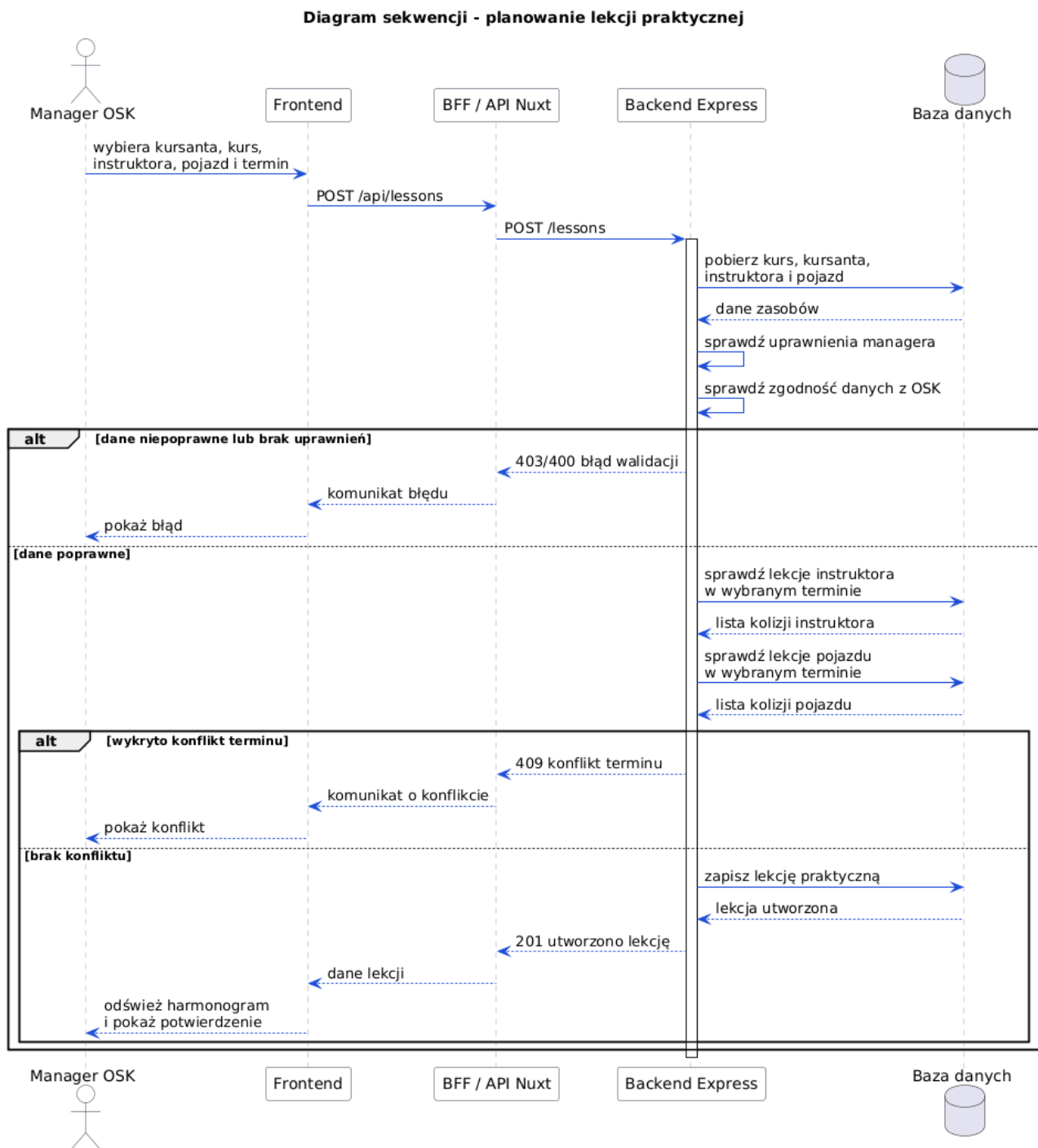
Źródło: Opracowanie własne

3.5. Diagramy sekwencji

Diagramy sekwencji przedstawiają wymianę komunikatów pomiędzy uczestnikami procesu w kolejności czasowej. W dokumentacji systemu OSK-Manager przygotowano dwa diagramy odpowiadające kluczowym scenariuszom opisanym wcześniej: zaplanowaniu lekcji praktycznej oraz ocenie zakończonej lekcji. Diagramy pokazują uproszczoną komunikację pomiędzy użytkownikiem, interfejsem aplikacji, backendem oraz bazą danych.

3.5.1. Diagram sekwencji: zaplanowanie lekcji praktycznej

Pierwszy diagram przedstawia komunikację realizowaną podczas tworzenia lekcji praktycznej przez managera OSK. Szczególnie istotne są zapytania sprawdzające dostępność instruktora i pojazdu, ponieważ od ich wyniku zależy zapis lekcji.

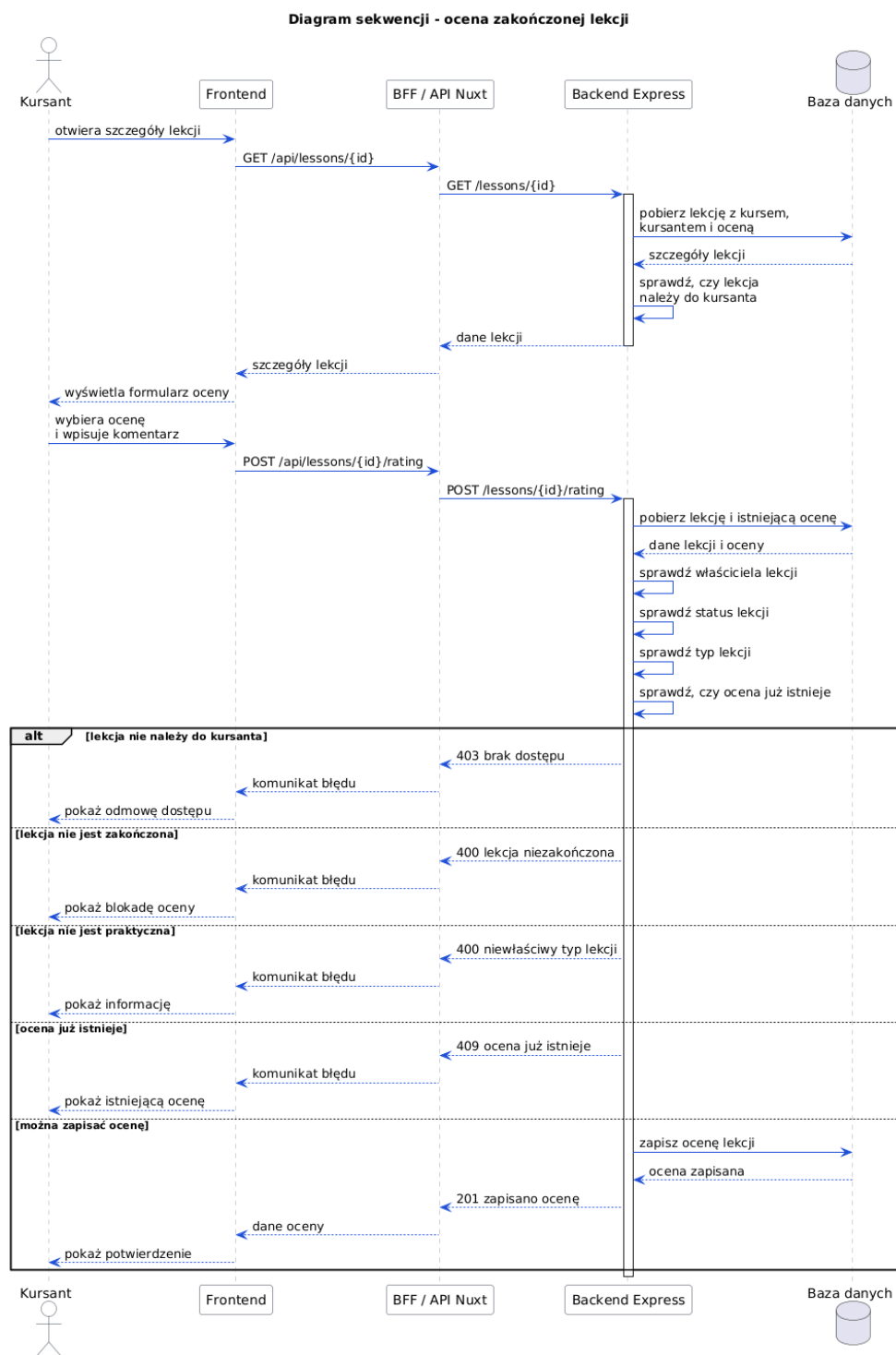


Rys. 3.5. Diagram sekwencji procesu planowania lekcji praktycznej

Źródło: Opracowanie własne

3.5.2. Diagram sekwencji: ocena zakończonej lekcji

Drugi diagram przedstawia komunikację podczas wystawiania oceny zakończonej lekcji przez kursanta. Backend sprawdza, czy lekcja należy do zalogowanego kursanta, czy jest zakończona, czy jest lekcją praktyczną oraz czy nie ma już zapisanej oceny.



Rys. 3.6. Diagram sekwencji procesu oceny zakończonej lekcji

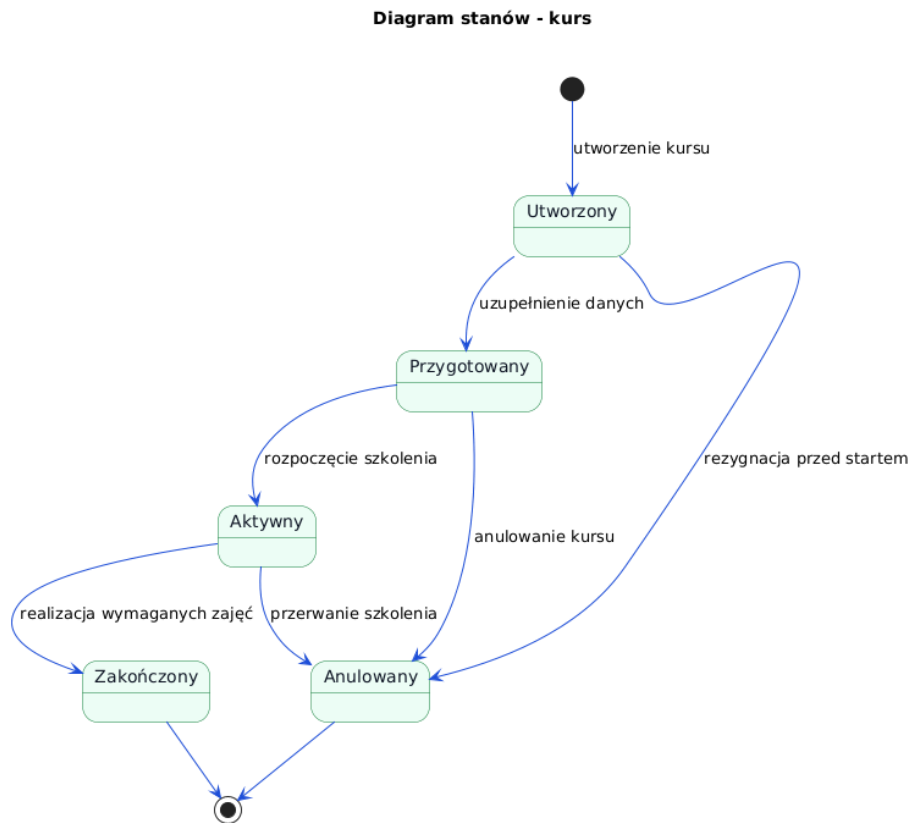
Źródło: Opracowanie własne

3.6. Diagramy stanów

Diagramy stanów pokazują możliwe stany wybranych obiektów systemu oraz zdarzenia powodujące przejścia pomiędzy nimi. W systemie OSK-Manager szczególnie istotne są cykle życia kursu oraz lekcji, ponieważ wpływają one na harmonogram, dostępność zasobów, panel kursanta oraz rozliczenia.

3.6.1. Diagram stanów: kurs

Pierwszy diagram przedstawia uproszczony cykl życia kursu. Kurs po utworzeniu jest przygotowywany organizacyjnie, następnie przechodzi w stan aktywny, a po realizacji wymaganych zajęć może zostać zakończony. W przypadku rezygnacji kursanta lub decyzji menedżera kurs może zostać anulowany.



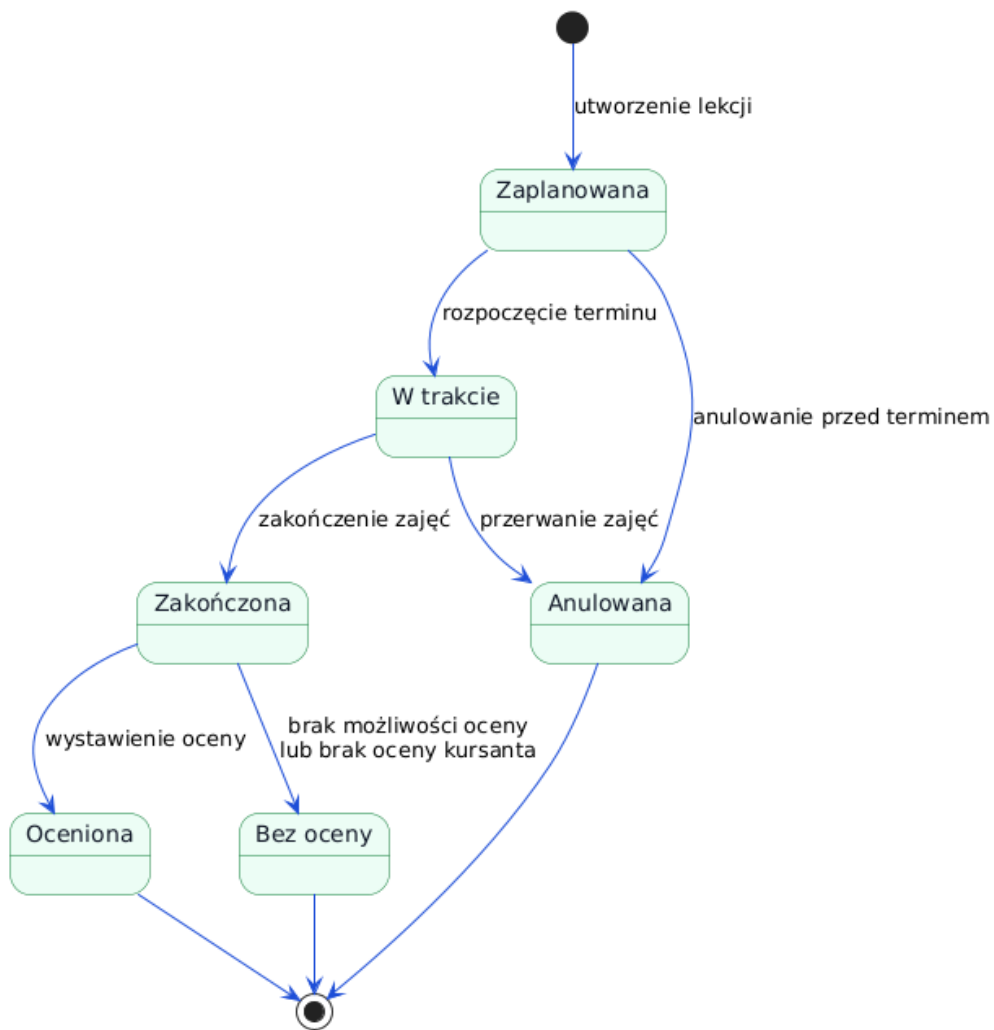
Rys. 3.7. Diagram stanów kursu w systemie OSK-Manager

Źródło: Opracowanie własne

3.6.2. Diagram stanów: lekcja

Drugi diagram przedstawia cykl życia lekcji. Lekcja po zaplanowaniu oczekuje na termin realizacji. Następnie może zostać przeprowadzona i zakończona albo anulowana przed rozpoczęciem. Dla zakończonej lekcji praktycznej możliwe jest zapisanie oceny przez kursanta.

Diagram stanów - lekcja



Rys. 3.8. Diagram stanów lekcji w systemie OSK-Manager

Źródło: Opracowanie własne

4. Technologie i wymagania sprzętowe

Niniejszy rozdział opisuje technologie wykorzystane w systemie OSK-Manager oraz podstawowe wymagania potrzebne do uruchomienia aplikacji. Projekt ma charakter aplikacji webowej z oddzielną częścią kliencką, warstwą pośredniczącą oraz backendem komunikującym się z bazą danych.

4.1. Technologie frontendowe

Część frontendowa systemu została przygotowana jako aplikacja oparta na frameworku *Nuxt*. Framework ten bazuje na bibliotece *Vue* i pozwala tworzyć aplikacje typu single page application oraz aplikacje renderowane po stronie serwera. W projekcie zastosowano język *TypeScript*, dzięki czemu kod interfejsu użytkownika korzysta z typowania statycznego i jest łatwiejszy w utrzymaniu.

Za warstwę wizualną odpowiada *Tailwind CSS*. Umożliwia on budowanie spójnych widoków na podstawie klas narzędziowych, bez konieczności tworzenia rozbudowanych arkuszy stylów dla każdego elementu osobno. W projekcie wykorzystano również komponenty zbudowane wokół biblioteki *shadcn-nuxt* oraz elementów interfejsu *Reka UI*. Ikony w aplikacji pochodzą z biblioteki *lucide-vue-next*.

Frontend posiada także własną warstwę komunikacji z backendem. Część żądań przechodzi przez trasy serwerowe Nuxt, które pełnią funkcję BFF, czyli pośrednika między przeglądarką a właściwym API. Takie rozwiązanie ułatwia ukrycie szczegółów komunikacji z backendem, obsługę ciasteczek sesyjnych oraz wprowadzanie trybu demonstracyjnego lub mocków w środowisku deweloperskim.

4.2. Technologie backendowe

Backend systemu został przygotowany w środowisku *Node.js* z użyciem frameworka *Express*. Kod backendu również napisano w języku *TypeScript*. Express odpowiada za obsługę tras HTTP, rejestrowanie modułów API, obsługę ciasteczek, konfigurację CORS oraz przekazywanie błędów do wspólnego mechanizmu obsługi wyjątków.

Dostęp do danych realizowany jest przez *Prisma*. Prisma pełni rolę warstwy ORM i udostępnia typowany klient do komunikacji z bazą danych. Schemat bazy danych jest opisany w pliku `schema.prisma`, co ułatwia utrzymywanie relacji między encjami oraz generowanie klienta używanego w usługach backendowych.

Do walidacji danych wejściowych użyto biblioteki *Zod*. Schematy walidacyjne pozwalają sprawdzać poprawność danych przekazywanych do API niezależnie od walidacji wykonywanej w interfejsie użytkownika. Projekt wykorzystuje również mechanizmy *OpenAPI* i *Swagger UI*, dzięki którym możliwe jest dokumentowanie oraz testowanie kontraktu API.

4.3. Baza danych i usługi zewnętrzne

System korzysta z relacyjnej bazy danych *PostgreSQL*. Projekt został przystosowany do pracy z bazą Supabase Postgres, dlatego konfiguracja backendu wymaga connection stringa do bazy oraz kluczy Supabase. Supabase pełni w projekcie rolę dostawcy bazy danych oraz usług pomocniczych, takich jak przechowywanie plików lub integracja z mechanizmami autoryzacji.

Wrażliwe dane konfiguracyjne, takie jak adres bazy danych, klucze Supabase oraz adres backendu, powinny być przechowywane w plikach środowiskowych. Klucze serwisowe nie powinny trafiać do kodu frontendu ani do publicznego repozytorium.

4.4. Narzędzia deweloperskie

W projekcie wykorzystywane są narzędzia wspierające jakość kodu. *ESLint* służy do statycznej analizy kodu i wykrywania potencjalnych błędów stylistycznych lub składniowych. *Prettier* odpowiada za formatowanie kodu. Testy automatyczne uruchamiane są przy pomocy *Vitest*. W części frontendowej dostępny jest również *vue-tsc*, który pozwala sprawdzać poprawność typów w komponentach Vue.

Backend udostępnia skrypty do uruchamiania aplikacji w trybie deweloperskim, budowania wersji produkcyjnej, generowania klienta Prisma, uruchamiania migracji, lintowania oraz testowania. Frontend udostępnia analogiczne skrypty dla aplikacji Nuxt, w tym uruchamianie trybu developerskiego, budowanie aplikacji, generowanie typów API oraz testy.

4.5. Wymagania sprzętowe i środowiskowe

Do uruchomienia aplikacji w środowisku deweloperskim wymagany jest komputer z zainstalowanym środowiskiem *Node.js* oraz menedżerem pakietów *npm*. Wymagany jest również dostęp do instancji bazy danych PostgreSQL, w projekcie przyjęto Supabase Postgres. Aplikacja backendowa domyślnie działa na porcie 3001, natomiast frontend uruchamiany jest jako aplikacja Nuxt na porcie wskazanym przez środowisko developerskie.

Minimalne wymagania środowiskowe obejmują:

- środowisko Node.js zgodne z używanymi zależnościami projektu,
- zainstalowane pakiety npm dla części frontendowej i backendowej,
- dostęp do bazy PostgreSQL,
- poprawnie skonfigurowane zmienne środowiskowe,
- przeglądarkę internetową do korzystania z aplikacji.

W środowisku produkcyjnym backend i frontend powinny być uruchomione na stabilnym serwerze lub platformie hostingowej z obsługą aplikacji Node.js. Baza danych powinna być zabezpieczona hasłem, połączeniem szyfrowanym oraz ograniczeniem dostępu do kluczy serwisowych.

5. Struktura systemu

System OSK-Manager został zaprojektowany jako aplikacja webowa z podziałem na kilka logicznych warstw. Taki podział ułatwia rozwój systemu, testowanie poszczególnych modułów oraz oddzielenie logiki prezentacji od logiki biznesowej i dostępu do danych.

5.1. Ogólna architektura

Architektura systemu składa się z następujących elementów:

- warstwy frontendowej, odpowiedzialnej za interfejs użytkownika,
- warstwy BFF w aplikacji Nuxt, odpowiedzialnej za pośredniczenie w komunikacji z API,
- backendu Express, odpowiedzialnego za realizację logiki biznesowej,
- warstwy usług i walidacji, odpowiedzialnej za reguły domenowe,
- warstwy dostępu do danych opartej o Prisma,
- relacyjnej bazy danych PostgreSQL.

Użytkownik korzysta z aplikacji przez przeglądarkę internetową. Interfejs wysyła żądania do tras serwerowych Nuxt albo bezpośrednio do API, zależnie od konfiguracji. Backend odbiera żądanie, sprawdza sesję użytkownika, waliduje dane wejściowe, wykonuje logikę biznesową w odpowiednim serwisie i zapisuje lub odczytuje dane z bazy.

5.2. Warstwa frontendowa

Frontend odpowiada za prezentację danych oraz obsługę interakcji użytkownika. W projekcie zastosowano strukturę opartą na stronach, komponentach, kompozycjach i typach. Strony odpowiadają konkretnym widokom aplikacji, na przykład listom kursantów, harmonogramowi, szczegółom instruktora lub panelowi konta. Komponenty wielokrotnego użytku obsługują formularze, tabele, kalendarze, dialogi i elementy nawigacyjne.

Istotną częścią frontendu są kompozycje, które wydzielają logikę pobierania danych, filtrowania, paginacji oraz obsługi formularzy poza same komponenty widoku. Dzięki temu kod interfejsu pozostaje czytelniejszy, a operacje takie jak pobranie listy pojazdów, zapis kursanta albo odświeżenie harmonogramu mogą być używane w spójny sposób w wielu miejscach aplikacji.

5.3. Warstwa BFF

W aplikacji frontendowej znajduje się warstwa BFF, czyli Backend for Frontend. Jest ona realizowana przez trasy serwerowe Nuxt. Jej zadaniem jest dopasowanie komunikacji do potrzeb interfejsu użytkownika oraz ukrycie części szczegółów backendu przed przeglądarką.

Warstwa BFF obsługuje między innymi przekazywanie żądań do backendu, normalizację odpowiedzi, walidację parametrów żądań oraz tryb mock dla wybranych danych w środowisku demonstracyjnym. Dzięki temu frontend może korzystać z jednolitego sposobu komunikacji, nawet jeżeli część danych pochodzi z backendu, a część z adapterów testowych.

5.4. Warstwa backendowa

Backend został podzielony na trasy, kontrolery, serwisy, schematy walidacji oraz moduły pomocnicze. Trasy określają adresy endpointów HTTP. Kontrolery odpowiadają za odbiór żądania, wywołanie odpowiedniej usługi i zwrócenie odpowiedzi. Serwisy zawierają właściwą logikę biznesową, na przykład sprawdzanie dostępności instruktora, zapis lekcji, obsługę płatności lub walidację przynależności danych do konkretnego OSK.

W systemie można wyróżnić następujące główne moduły backendu:

- moduł autoryzacji i profilu użytkownika,
- moduł ośrodków szkolenia kierowców,
- moduł kursantów,
- moduł instruktorów,
- moduł pojazdów,
- moduł typów kursów i kursów,
- moduł lekcji,
- moduł wydarzeń instruktorskich,
- moduł harmonogramu,
- moduł ocen lekcji,
- moduł płatności i planów płatności.

5.5. Opis bazy danych

Baza danych systemu została zaprojektowana jako relacyjny model danych. Centralnym elementem modelu jest użytkownik, który może pełnić jedną z ról: kursanta, instruktora, managera lub administratora. Dane wspólne użytkownika przechowywane są w encji użytkownika, natomiast dane specyficzne dla roli są wydzielone do profilu kursanta lub profilu instruktora.

Ośrodek szkolenia kierowców grupuje kursantów, instruktorów, pojazdy, kursy oraz ustawienia organizacyjne. Dzięki temu system może obsługiwać wiele szkół jazdy i jednocześnie kontrolować, czy użytkownik wykonuje operacje wyłącznie na danych powiązanych z właściwym ośrodkiem.

Kurs jest powiązany z ośrodkiem, typem kursu oraz uczestnikami. Uczestnictwo kursanta w kursie opisuje osobna encja, która pozwala przechowywać status udziału kursanta. Lekcja jest przypisana do kursu, kursanta i instruktora, a w przypadku lekcji praktycznej może być dodatkowo powiązana z pojazdem. Taki model pozwala sprawdzać, czy lekcja została zaplanowana dla właściwego kursanta, właściwego kursu i właściwej szkoły.

Pojazdy są przypisane do ośrodka i posiadają dane takie jak nazwa, numer rejestracyjny, marka, model, rok produkcji, przebieg, zdjęcie oraz status dostępności. Pojazd może być używany w lekcjach praktycznych oraz wydarzeniach instruktorskich. System przechowuje również informacje o domyślnym pojeździe szkoły.

Dostępność instruktorów jest modelowana przez kilka powiązanych encji. Domyślne godziny pracy określają typowy tygodniowy grafik instruktora. Wyjątki dzienne pozwalają zmienić godziny pracy w konkretnym dniu. Blokady czasu opisują przerwy, spotkania lub inne zajęcia, natomiast urlopy określają dłuższe okresy niedostępności. Na podstawie tych danych oraz już istniejących lekcji system może wyliczać wolne terminy.

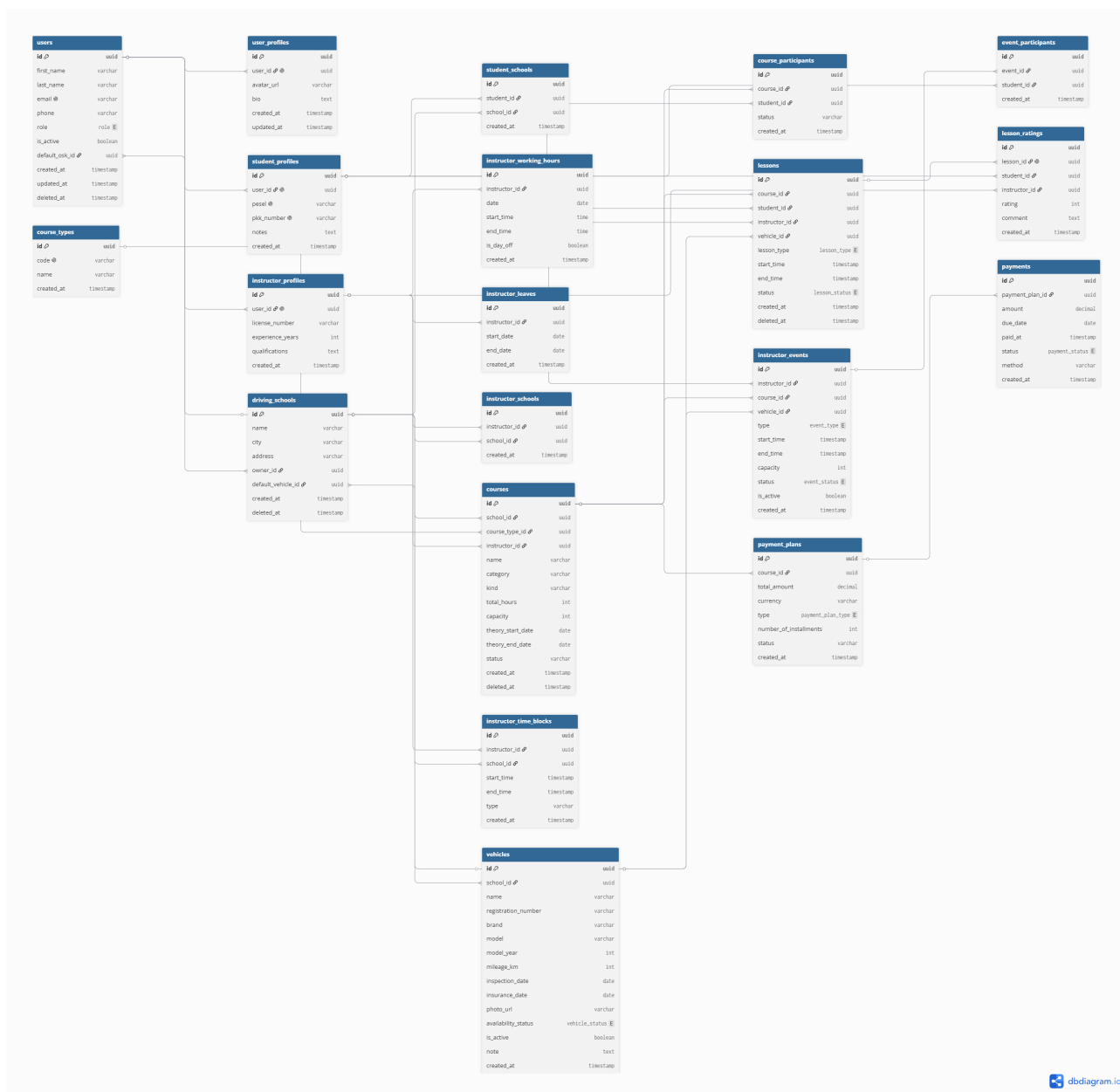
Wydarzenia instruktorskie opisują zajęcia typu jazda lub teoria. Mogą posiadać uczestników, powiązany kurs, pojazd, pojemność i status. Dzięki temu system obsługuje nie tylko pojedyncze lekcje, lecz także zajęcia grupowe lub inne zdarzenia widoczne w harmonogramie instruktora.

Rozliczenia kursu opisują plany płatności oraz płatności. Plan płatności przechowuje łączną kwotę, walutę, typ płatności i liczbę rat, natomiast płatność opisuje konkretną kwotę, termin płatności, datę opłacenia, status i metodę płatności.

Oceny lekcji przechowywane są w osobnej encji powiązanej z lekcją, kursantem oraz instruktorem. Dzięki temu kursant może ocenić zakończoną lekcję praktyczną, a manager lub instruktor może analizować informacje zwrotne dotyczące jakości szkolenia.

5.6. Diagram ERD

Na rysunku 5.1 przedstawiono diagram ERD bazy danych systemu OSK-Manager. Diagram pokazuje najważniejsze encje, ich atrybuty oraz relacje między użytkownikami, profilami, szkołami jazdy, kursami, lekcjami, wydarzeniami, pojazdami, ocenami i płatnościami.



Rys. 5.1. Diagram ERD bazy danych systemu OSK-Manager

Źródło: Opracowanie własne

5.7. Integralność danych

Model danych zawiera relacje i ograniczenia pomagające zachować spójność informacji. Przykładowo adres e-mail użytkownika jest unikalny, numer rejestracyjny pojazdu jest unikalny w ramach szkoły, a przypisanie kursanta do kursu nie powinno się powtarzać. W bazie występują także indeksy przyspieszające wyszukiwanie danych w harmonogramie, na przykład po instruktorze, pojeździe, kursancie i czasie rozpoczęcia lekcji.

Sama struktura bazy nie zastępuje logiki biznesowej. Dlatego część reguł jest realizowana w serwisach backendowych. Dotyczy to przede wszystkim sprawdzania kolizji terminów, zgodności kursanta, instruktora i pojazdu z tym samym OSK oraz zasad dotyczących oceniania lekcji.

5.8. Diagram klas

Diagram klas systemu został przedstawiony wcześniej w rozdziale dotyczącym modelowania systemu, na rysunku 3.2. W kontekście struktury systemu pełni on rolę uproszczonego widoku domeny i pokazuje najważniejsze klasy biznesowe: użytkownika, ośrodek szkolenia, kurs, lekcję, pojazd, dostępność instruktora oraz płatność.

Właściwy model implementacyjny jest bardziej szczegółowy niż uproszczony diagram klas. W kodzie i schemacie bazy danych znajdują się dodatkowe encje pomocnicze, takie jak profile użytkowników, ustawienia szkoły, uczestnicy kursów, wydarzenia instruktorskie, uczestnicy wydarzeń, plany płatności i oceny lekcji. Diagram klas w dokumentacji pokazuje więc główny obraz domeny, natomiast opis bazy danych rozszerza go o szczegóły implementacyjne.

6. Projekt GUI

Interfejs użytkownika systemu OSK-Manager został zaprojektowany jako aplikacja webowa dostępna z poziomu przeglądarki. Głównym założeniem GUI jest szybki dostęp do najważniejszych obszarów pracy szkoły jazdy: kursantów, instruktorów, kursów, pojazdów, harmonogramu, płatności i ocen.

6.1. Założenia interfejsu

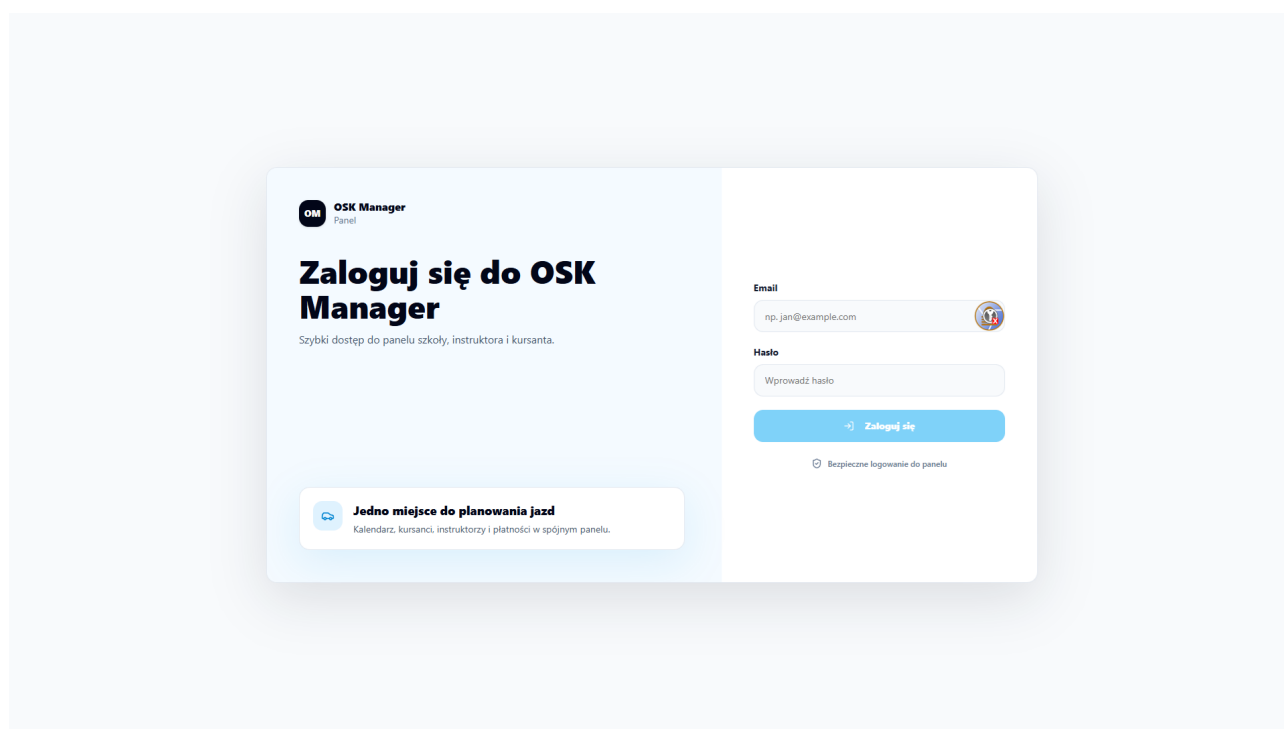
Aplikacja wykorzystuje układ panelowy. Po zalogowaniu użytkownik widzi obszar roboczy z nawigacją boczną, górnym paskiem oraz właściwą treścią strony. Nawigacja zależy od roli użytkownika. Manager otrzymuje dostęp do funkcji administracyjnych szkoły, instruktor do własnego harmonogramu i ocen, natomiast kursant do swoich kursów, lekcji, płatności i możliwości rezerwacji terminu.

Widoki systemu powinny być czytelne zarówno na komputerze, jak i na urządzeniach mobilnych. Listy i tabele są używane tam, gdzie użytkownik porównuje wiele rekordów, na przykład kursantów, instruktorów lub pojazdów. Kalendarze i siatki czasu są używane w obszarach harmonogramu. Formularze są dzielone na logiczne sekcje, aby ułatwić wprowadzanie danych.

6.2. Widok logowania

Widok logowania umożliwia użytkownikowi podanie adresu e-mail oraz hasła. Formularz sprawdza, czy wymagane pola zostały uzupełnione, a następnie wysyła dane do mechanizmu autoryzacji. Po poprawnym zalogowaniu użytkownik zostaje przekierowany do aplikacji. Jeżeli użytkownik był już zalogowany, system może przekierować go bezpośrednio do właściwego widoku.

W środowisku demonstracyjnym aplikacja może udostępniać przyciski uzupełniające pola przykładowymi danymi. Mechanizm ten ma charakter pomocniczy i nie zastępuje właściwego procesu logowania.



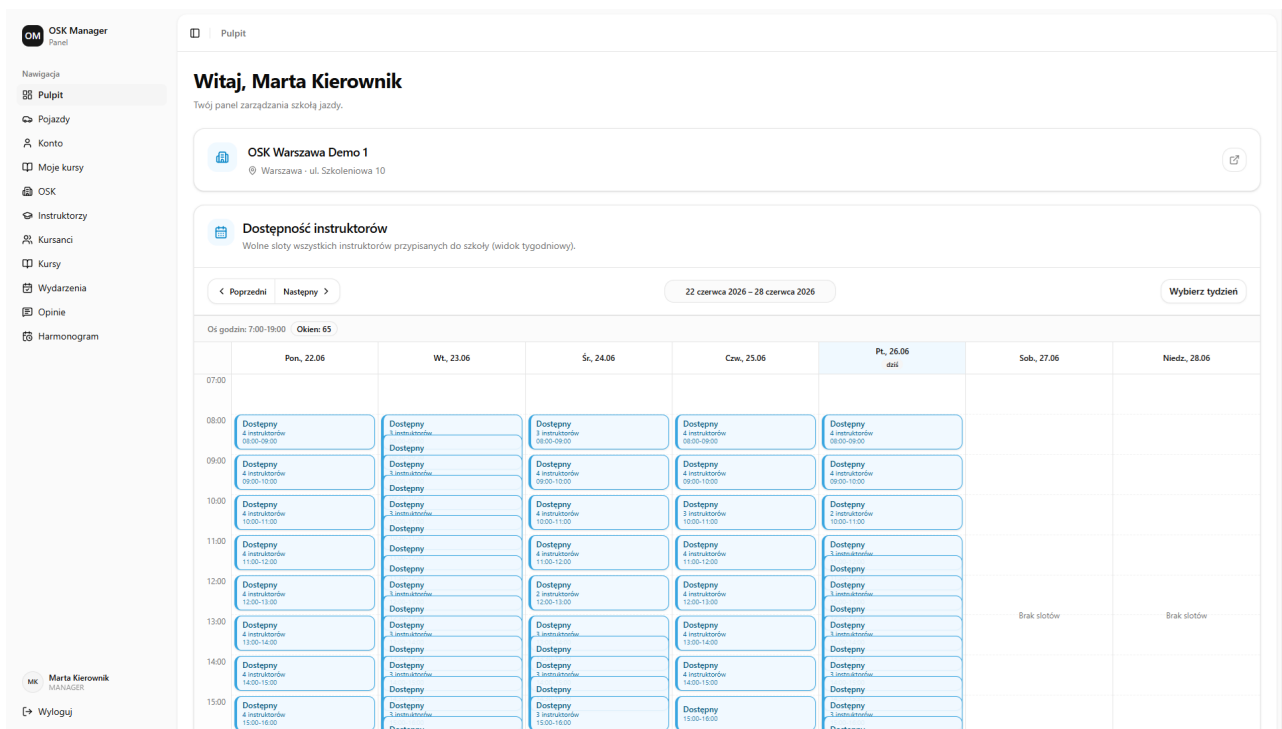
Rys. 6.1. Widok logowania do systemu OSK-Manager

Źródło: Opracowanie własne

6.3. Dashboard

Dashboard jest pierwszym widokiem po zalogowaniu. Dla managera prezentuje kontekst domyślnego ośrodka szkolenia kierowców oraz skrót do harmonogramu i dostępności instruktorów. Jeżeli manager nie ma ustawionego domyślnego OSK, aplikacja prowadzi go do konfiguracji ośrodka.

Dla kursanta i instruktora dashboard pełni funkcję widoku startowego. Może prezentować przypisane szkoły, najważniejsze odnośniki oraz informacje potrzebne do przejścia do kursów, lekcji albo harmonogramu.



Rys. 6.2. Dashboard managera z dostępnością instruktorów

Źródło: Opracowanie własne

6.4. Panel managera

Panel managera obejmuje widoki przeznaczone do zarządzania szkołą jazdy. Najważniejsze obszary to:

- lista i szczegóły kursantów,
- lista i szczegóły instruktorów,
- zarządzanie ośrodkami szkolenia kierowców,
- zarządzanie kursami,
- zarządzanie pojazdami,
- harmonogram szkoły,
- edycja lekcji i wydarzeń,
- przegląd ocen lekcji.

Widok kursantów pozwala przeglądać listę osób przypisanych do szkoły, filtrować dane, dodać kursanta oraz przypisać go do kursu. Szczegóły kursanta obejmują dane profilu, status procesu szkolenia, notatki, harmonogram lekcji, płatności i kursy.

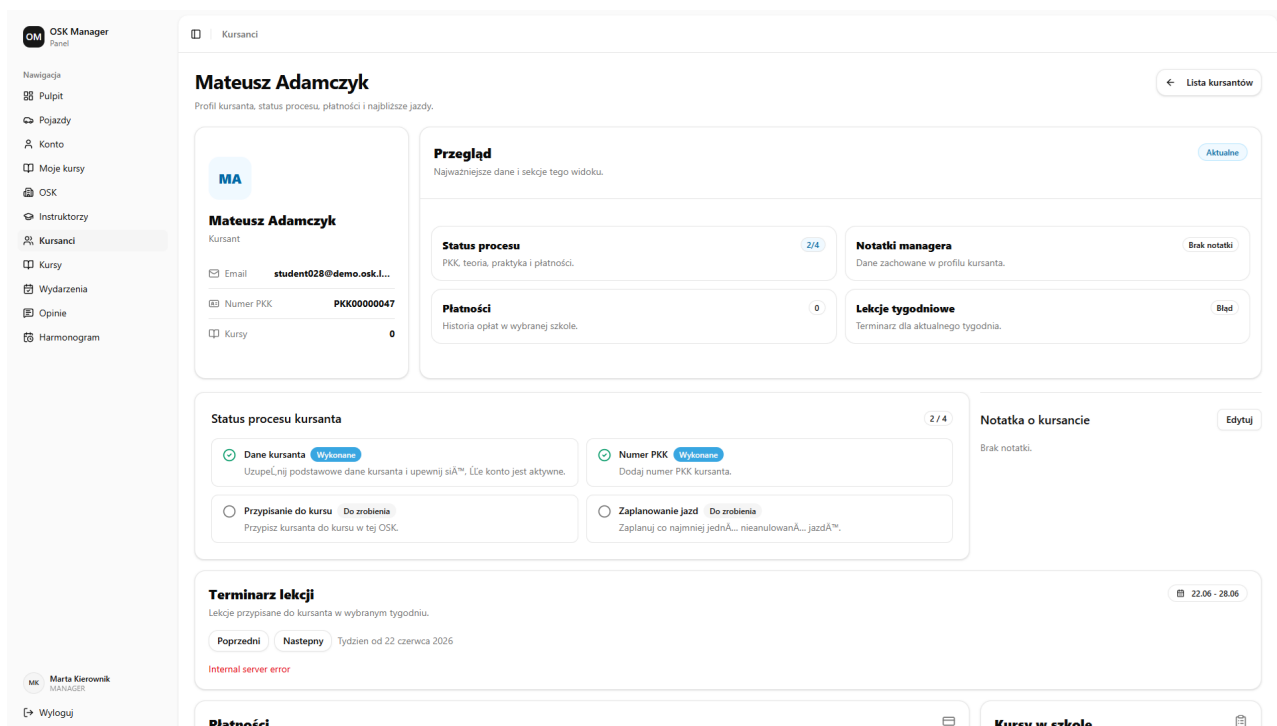
Widok instruktorów pozwala dodawać instruktorów, przeglądać ich dane, kwalifikacje, dostępność, harmonogram i oceny. Manager może przejść do edycji dostępności, sprawdzić tygodniowy grafik oraz zarządzać blokadami czasu.

Widok pojazdów pozwala przeglądać flotę szkoły, dodawać pojazdy, edytować ich dane, oznaczać status dostępności i przechowywać podstawowe informacje eksploatacyjne, takie jak numer rejestracyjny, marka, model, przebieg, przegląd i ubezpieczenie.

Widok harmonogramu prezentuje zajęcia szkoły w ujęciu czasowym. Manager może analizować obłożenie instruktorów, sprawdzać zaplanowane lekcje oraz przechodzić do edycji wydarzeń. Harmonogram jest jednym z kluczowych widoków systemu, ponieważ łączy dane kursantów, instruktorów, pojazdów i kursów.

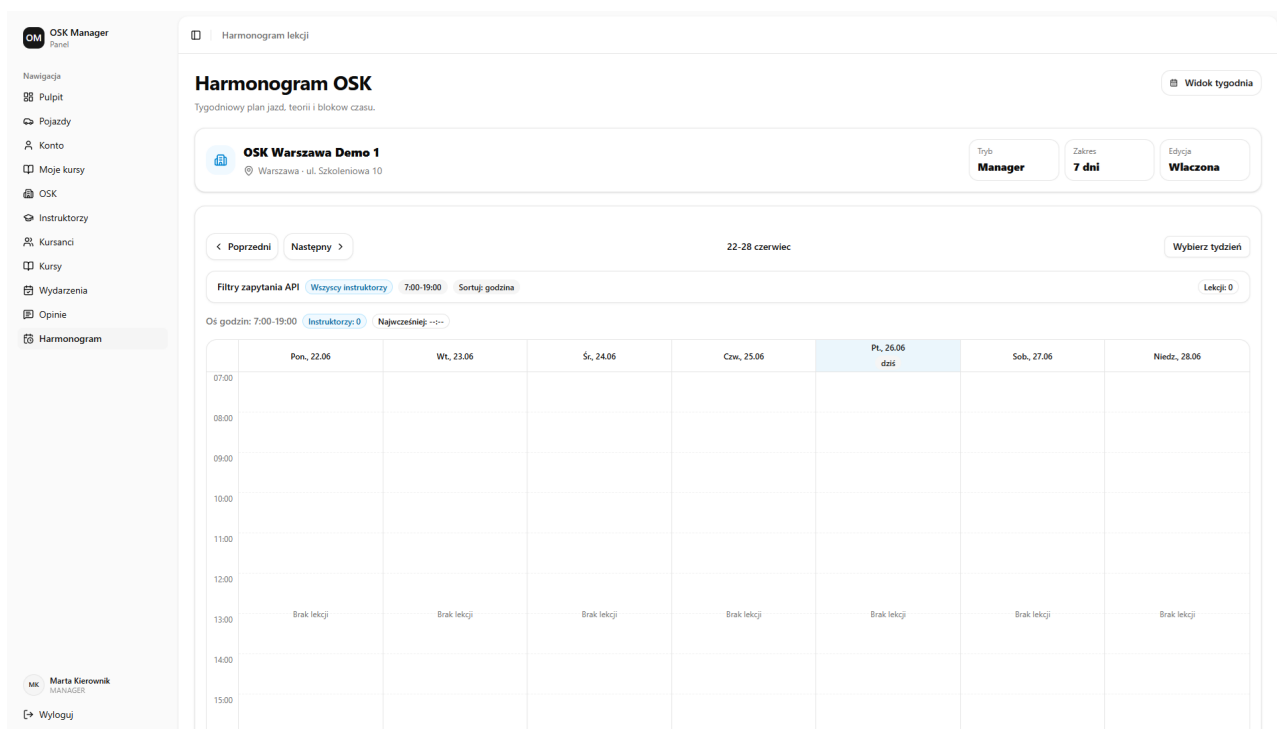
Rys. 6.3. Widok listy kursantów w panelu menadżera

Źródło: Opracowanie własne



Rys. 6.4. Widok szczegółów kursanta w panelu menagera

Źródło: Opracowanie własne



Rys. 6.5. Widok harmonogramu OSK w panelu menagera

Źródło: Opracowanie własne

6.5. Widoki kursanta

Kursant korzysta z widoków związanych z własnym szkoleniem. Może przeglądać swoje kursy, zaplanowane lekcje, płatności oraz oceny. Widok rezerwacji lekcji pozwala wybrać kurs, tydzień oraz dostępny termin, a następnie zapisać lekcję, jeżeli spełnione są warunki dostępności.

W widoku lekcji kursant może sprawdzić nadchodzące i zakończone zajęcia. Dla zakończonych lekcji praktycznych dostępny jest mechanizm wystawienia oceny. System blokuje ocenę lekcji, jeżeli nie została zakończona, nie jest lekcją praktyczną albo została już wcześniej oceniona.

6.6. Widoki instruktora

Instruktor korzysta przede wszystkim z widoków harmonogramu oraz ocen. Może sprawdzić swoje lekcje i wydarzenia oraz zobaczyć opinie wystawione przez kursantów. Część widoków jest współdzielona z kursantem, ale prezentowane dane są filtrowane zgodnie z rolą użytkownika.

6.7. Wspólne elementy interfejsu

W wielu widokach powtarzają się te same wzorce interfejsu. Listy danych posiadają stany ładowania, puste stany oraz komunikaty błędów. Formularze pokazują walidację i blokują zapis w trakcie wysyłania danych. Dialogi służą do tworzenia, edycji lub potwierdzania operacji, na przykład usunięcia rekordu albo anulowania lekcji.

System wykorzystuje oznaczenia statusów w formie etykiet, przyciski akcji, selektory, filtry, pola daty i czasu oraz komponenty kalendarza. Dzięki temu użytkownik może wykonywać podobne operacje w podobny sposób w różnych częściach aplikacji.

7. Propozycja testów

Testowanie systemu OSK-Manager powinno obejmować zarówno logikę biznesową backendu, jak i poprawność działania interfejsu użytkownika. Szczególne znaczenie mają obszary związane z autoryzacją, planowaniem lekcji, dostępnością instruktorów, dostępnością pojazdów, płatnościami oraz ocenami lekcji.

7.1. Testy jednostkowe

Testy jednostkowe powinny sprawdzać pojedyncze funkcje i reguły domenowe bez konieczności uruchamiania całej aplikacji. W systemie OSK-Manager szczególnie istotne są testy:

- sprawdzania poprawności identyfikatorów UUID,
- walidacji statusów pojazdów,
- kwalifikacji instruktora do prowadzenia danego typu kursu,
- domyślnych godzin pracy instruktora,
- zasad rejestracji użytkowników i przypisywania ról,
- obsługi błędów integracji z Supabase.

Takie testy ograniczają ryzyko przypadkowej zmiany podstawowych reguł systemu podczas dalszego rozwoju aplikacji.

7.2. Testy usług backendowych

Najważniejsza logika systemu znajduje się w usługach backendowych. Testy tej warstwy powinny obejmować:

- tworzenie i edycję lekcji,
- anulowanie lekcji,
- rezerwację lekcji przez kursanta,
- sprawdzanie konfliktów harmonogramu,
- obsługę wydarzeń instruktorskich,
- wyliczanie dostępności instruktorów,
- obsługę płatności kursanta,
- zmianę statusu procesu szkolenia,
- wystawianie ocen zakończonych lekcji.

Testy powinny weryfikować zarówno scenariusze poprawne, jak i przypadki błędne. Przykładowo system powinien odrzucić próbę zaplanowania lekcji w terminie, w którym instruktor jest zajęty, pojazd jest niedostępny lub dane należą do innego OSK.

7.3. Testy API i kontraktu

Backend udostępnia API HTTP, dlatego należy testować poprawność endpointów oraz zgodność dokumentacji OpenAPI z implementacją. Testy kontraktu API powinny sprawdzać, czy najważniejsze ścieżki są opisane, czy odpowiedzi mają oczekiwaną strukturę oraz czy schematy walidacji odpowiadają rzeczywistym danym zwracanym przez backend.

Warto objąć testami następujące obszary API:

- logowanie, odświeżanie sesji i pobieranie danych użytkownika,
- operacje na kursantach,
- operacje na instruktorach,
- operacje na pojazdach,
- operacje na kursach i typach kursów,
- planowanie i edycję lekcji,
- harmonogram szkoły,
- oceny lekcji.

7.4. Testy warstwy BFF

Warstwa BFF po stronie Nuxt wymaga osobnych testów, ponieważ odpowiada za pośredniczenie między frontendem i backendem. Testy powinny sprawdzać wybór adaptera komunikacji, poprawność przekazywania żądań do backendu, obsługę błędów połączenia oraz walidację parametrów żądań.

Szczególnie ważne jest sprawdzenie, czy aplikacja poprawnie zachowuje się w przypadku braku konfiguracji backendu, błędnego adresu API lub niedostępnego upstreamu. Użytkownik powinien otrzymać czytelny komunikat błędu zamiast technicznego wyjątku.

7.5. Testy manualne GUI

Testy manualne powinny obejmować pełne scenariusze wykonywane z perspektywy użytkownika. Proponowane przypadki testowe to:

1. Zalogowanie się do systemu poprawnymi danymi.
2. Próba logowania z błędnym hasłem.
3. Utworzenie nowego kursanta przez managera.
4. Przypisanie kursanta do kursu.
5. Dodanie instruktora i ustawienie jego dostępności.
6. Dodanie pojazdu i zmiana jego statusu dostępności.
7. Zaplanowanie lekcji praktycznej z instruktorem i pojazdem.
8. Próba zaplanowania lekcji w terminie powodującym konflikt.

9. Anulowanie lekcji.
10. Wystawienie oceny zakończonej lekcji przez kursanta.
11. Sprawdzenie listy płatności kursanta.
12. Sprawdzenie harmonogramu szkoły w widoku managera.

Każdy test manualny powinien określać warunki początkowe, kroki wykonania oraz oczekiwany rezultat. Dzięki temu testy mogą być powtarzane po każdej większej zmianie w aplikacji.

7.6. Testy regresji

Testy regresji powinny być wykonywane po zmianach w logice harmonogramu, autoryzacji, modelu danych oraz formularzach. Szczególnie wrażliwe obszary to planowanie lekcji, sprawdzanie dostępności, przypisywanie danych do OSK oraz uprawnienia użytkowników.

Minimalny zestaw regresji powinien obejmować uruchomienie testów automatycznych backendu i frontendu, sprawdzenie typów TypeScript, lintowanie oraz ręczne przejście najważniejszych scenariuszy GUI. Pozwala to szybko wykryć błędy w miejscach, które nie były bezpośrednio zmieniane, ale zależą od wspólnej logiki systemu.

8. Podsumowanie

Celem projektu OSK-Manager było zaprojektowanie systemu wspierającego zarządzanie ośrodkiem szkolenia kierowców. Analiza problemu pokazała, że szkoły jazdy potrzebują narzędzia, które centralizuje dane kursantów, instruktorów, pojazdów, kursów, lekcji, płatności i harmonogramu. Brak takiego systemu zwiększa ryzyko błędów organizacyjnych, zwłaszcza podwójnych rezerwacji instruktorów lub pojazdów.

W dokumentacji opisano wymagania funkcjonalne i нефункционаłne systemu, a następnie przedstawiono modelowanie obejmujące diagram przypadków użycia, scenariusze przypadków użycia, diagramy aktywności, diagramy sekwencji, diagramy stanów oraz diagram klas. Modele te pokazują najważniejsze procesy systemu: planowanie lekcji praktycznej, ocenę zakończonej lekcji, cykl życia kursu oraz cykl życia lekcji.

Projekt systemu opiera się na aplikacji webowej z podziałem na frontend, warstwę BFF, backend oraz relacyjną bazę danych. Frontend odpowiada za wygodny interfejs użytkownika, backend za logikę biznesową i walidację, a baza danych za trwałe przechowywanie informacji. Taki podział pozwala rozwijać system modułowo i utrzymywać czytelne granice odpowiedzialności.

Najważniejszą częścią systemu jest harmonogram. Łączy on dane kursantów, instruktorów, kursów i pojazdów, dlatego wymaga szczególnej kontroli poprawności. System powinien uniemożliwiać planowanie lekcji w terminach konfliktowych, pilnować zgodności danych z właściwym OSK oraz pozwalać instruktorom i kursantom na szybki podgląd własnych zajęć.

Istotną zaletą projektu jest także uwzględnienie różnych ról użytkowników. Manager zarządza szkołą, instruktor korzysta z harmonogramu i dostępności, kursant śledzi własny przebieg szkolenia, a administrator może nadzorować dane systemowe. Dzięki temu każda grupa użytkowników otrzymuje dostęp do funkcji zgodnych ze swoimi zadaniami.

Projekt może być rozwijany w przyszłości o kolejne funkcje, takie jak rozbudowane raportowanie, automatyczne powiadomienia, integracje z płatnościami online, eksport dokumentów, panel administracyjny dla wielu szkół oraz bardziej zaawansowaną analitykę jakości szkolenia. Podstawowy model systemu tworzy jednak solidną bazę do dalszej rozbudowy, ponieważ obejmuje kluczowe obiekty domenowe i najważniejsze procesy działania ośrodka szkolenia kierowców.

Spis rysunków

3.1	Diagram przypadków użycia systemu OSK-Manager	9
3.2	Diagram klas systemu OSK-Manager	10
3.3	Diagram aktywności procesu planowania lekcji praktycznej	13
3.4	Diagram aktywności procesu oceny zakończonej lekcji	14
3.5	Diagram sekwencji procesu planowania lekcji praktycznej	15
3.6	Diagram sekwencji procesu oceny zakończonej lekcji	16
3.7	Diagram stanów kursu w systemie OSK-Manager	17
3.8	Diagram stanów lekcji w systemie OSK-Manager	18
5.1	Diagram ERD bazy danych systemu OSK-Manager	24
6.1	Widok logowania do systemu OSK-Manager	26
6.2	Dashboard managera z dostępnością instruktorów	27
6.3	Widok listy kursantów w panelu managera	28
6.4	Widok szczegółów kursanta w panelu managera	29
6.5	Widok harmonogramu OSK w panelu managera	29

Spis tabel

1.1 Porównanie OSK-Manager z wybranymi rozwiązaniami	5
--	---

Spis listingów

Bibliografia

Wyższa Szkoła Informatyki i Zarządzania z siedzibą w Rzeszowie
Kolegium Informatyki Stosowanej

Streszczenie pracy dyplomowej

Tytuł pracy w języku polskim

Autor:

Promotor:

Promotor pomocniczy:

Słowa kluczowe: (nie więcej niż 200 znaków)

Treść streszczenia, czyli kilka zdań dotyczących treści pracy dyplomowej w języku polskim.

Abstract

**The University of Information Technology and Management in Rzeszów
Faculty of Applied Information Technology**

Diploma Thesis Summary

Tytuł pracy w języku angielskim

Author:

Supervisor:

Assistant supervisor:

Key words:

Treść streszczenia, czyli kilka zdań dotyczących treści pracy dyplomowej w języku polskim.